Project Documentation

Generated from repository documentation files.

Project README

MyRestaurant - Full-Stack Restaurant Ordering System

A comprehensive, production-ready restaurant application built with modern web technologies. This app enables customers to browse menus, place orders, reserve tables, and purchase gift cards, while providing admin and rider dashboards for operational management.

Features

Customer Features

- **User Authentication**: Secure registration and login using Firebase Auth with session cookies
- **Menu Browsing**: Interactive menu with categories, pricing, and high-quality images
- **Shopping Cart**: Persistent local cart with quantity management
- **Order Placement**: Seamless checkout with delivery details and order history
- **Table Reservations**: Book tables for dine-in experiences
- **Gift Cards**: Purchase and send digital gift cards
- **Contact Form**: Submit inquiries and feedback
- **Responsive Design**: Mobile-first UI with dark/light theme support

Admin Features

- **Order Management**: View, update, and track order statuses (pending → confirmed → out for delivery → delivered)
- **Dashboard Overview**: Real-time order pipeline and operational insights
- **Order Type Distinction**: Identifies food orders vs. gift card purchases

Rider Features

- **Dispatch Management**: Update order statuses for delivery tracking
- **Order History**: View assigned deliveries and completion status

Technical Features

- **Image Management**: Cloudinary integration for menu image storage and optimization
- **Database**: PostgreSQL with Prisma ORM for robust data management
- **Authentication**: Firebase Auth with server-side session validation
- **API Routes**: RESTful endpoints for all operations
- **Type Safety**: Full TypeScript implementation
- **Performance**: Optimized with Next.js App Router and Vercel deployment

Tech Stack

- **Frontend**: Next.js 16 (App Router), React 19, TypeScript
- **Styling**: CSS Modules with custom design system
- **Backend**: Next.js API Routes
- **Database**: PostgreSQL with Prisma ORM
- **Authentication**: Firebase Auth
- **Image Storage**: Cloudinary
- **Deployment**: Vercel (standalone output)
- **Package Manager**: pnpm

Prerequisites

Before running this project, ensure you have the following installed:

- **Node.js**: Version 18 or higher
- **pnpm**: Package manager (install with `npm install -g pnpm`)
- **PostgreSQL**: Database server (local or cloud instance)
- **Firebase Project**: For authentication
- **Cloudinary Account**: For image management

Installation

1. **Clone the repository**:

`bash`

```
git clone <repository-url>
cd project-3
```

1. **Install dependencies**:

```
bash
pnpm install
```

1. **Set up environment variables**:

```
Copy .env.example to .env.local and fill in the required values:
bash
cp .env.example .env.local
Required environment variables:
- DATABASE_URL: PostgreSQL connection string
- NEXT_PUBLIC_FIREBASE_API_KEY: Firebase API key
- NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN: Firebase auth domain
- NEXT_PUBLIC_FIREBASE_PROJECT_ID: Firebase project ID
- FIREBASE_CLIENT_EMAIL: Firebase service account email
- FIREBASE_PRIVATE_KEY: Firebase service account private key
- CLOUDINARY_CLOUD_NAME: Cloudinary cloud name
- CLOUDINARY_API_KEY: Cloudinary API key
- CLOUDINARY_API_SECRET: Cloudinary API secret
```

Database Setup

1. **Generate Prisma client**:

```
bash
```

```
pnpm db:generate
```

1. **Push database schema**:

```
bash

pnpm db:push
```

1. **Seed the database** (optional, adds sample data):

```
bash

pnpm db:seed
```

Running the Application

1. **Start development server**:

```
bash

pnpm dev
```

1. **Open your browser** and navigate to `http://localhost:3000`

Usage

For Customers

1. **Register/Login**: Create an account or sign in
2. **Browse Menu**: View available items by category
3. **Add to Cart**: Select items and quantities
4. **Checkout**: Provide delivery details and place order
5. **Track Orders**: View order history and status
6. **Reserve Tables**: Book tables for dine-in
7. **Purchase Gift Cards**: Select value and send to recipients

For Admins

1. **Access Dashboard**: Navigate to `/admin/dashboard`
2. **Manage Orders**: Update order statuses and view details
3. **Monitor Operations**: Track order pipeline

For Riders

1. **Access Dashboard**: Navigate to `/rider/dashboard`
2. **Update Deliveries**: Change order status as deliveries progress

API Endpoints

Authentication

- `GET /api/auth/me` - Get current user session
- `POST /api/auth/login` - User login
- `POST /api/auth/logout` - User logout
- `POST /api/auth/register` - User registration

Orders

- `GET /api/orders` - List orders (filtered by user role)
- `POST /api/orders` - Create new order
- `PUT /api/orders/[id]` - Update order status

Bookings

- `GET /api/bookings` - List bookings
- `POST /api/bookings` - Create table reservation

Menu

- `GET /api/menu` - List menu items

Contact

- `POST /api/contact` - Submit contact form

Uploads

- `POST /api/uploads/signature` - Upload signature images

Database Schema

User

- `id`: String (Primary Key)

- `firebaseUid`: String
- `name`: String
- `email`: String
- `role`: UserRole (customer/admin/rider)
- `createdAt`: DateTime

MenuItem

- `id`: String (Primary Key)
- `slug`: String
- `name`: String
- `description`: String
- `price`: Float
- `category`: MenuCategory
- `prepTime`: String
- `calories`: Int (Optional)
- `imagePublicId`: String
- `imageUrl`: String
- `featured`: Boolean (Optional)

Order

- `id`: String (Primary Key)
- `userId`: String (Foreign Key to User)
- `customerName`: String
- `customerEmail`: String
- `deliveryAddress`: String
- `notes`: String (Optional)
- `total`: Float
- `status`: OrderStatus
- `createdAt`: DateTime
- `items`: OrderItem[]

Booking

- `id`: String (Primary Key)
- `name`: String
- `email`: String
- `date`: String
- `time`: String

- `guests`: Int
- `notes`: String (Optional)
- `createdAt`: DateTime

Contact

- `id`: String (Primary Key)
- `name`: String
- `email`: String
- `message`: String
- `createdAt`: DateTime

Deployment

This app is configured for deployment on Vercel:

1. **Connect Repository**: Link your GitHub repository to Vercel
2. **Set Environment Variables**: Configure all required env vars in Vercel dashboard
3. **Deploy**: Vercel will automatically build and deploy using the standalone output

Build Commands

- **Build**: `pnpm build`
- **Start**: `pnpm start`

Scripts

- `pnpm dev` - Start development server
- `pnpm build` - Build for production
- `pnpm start` - Start production server
- `pnpm lint` - Run ESLint
- `pnpm typecheck` - Run TypeScript type checking
- `pnpm db:generate` - Generate Prisma client
- `pnpm db:push` - Push database schema
- `pnpm db:seed` - Seed database with sample data

Project Structure

```
project-3/
├── app/                    # Next.js App Router pages
│   ├── admin/dashboard/    # Admin dashboard
│   ├── api/                # API routes
│   ├── booking/            # Table reservation page
│   ├── checkout/           # Order checkout
│   ├── contact/            # Contact form
│   ├── giftcards/          # Gift card purchase
│   ├── login/              # User login
│   ├── menu/               # Menu browsing
│   ├── orders/             # Order history
│   ├── register/           # User registration
│   └── rider/dashboard/    # Rider dashboard
├── components/             # Reusable React components
├── lib/                    # Utility functions and configurations
├── prisma/                 # Database schema and migrations
├── public/                 # Static assets
└── styles/                 # Additional CSS files
```

Contributing

1. Fork the repository
2. Create a feature branch: `git checkout -b feature/your-feature`
3. Make your changes and commit: `git commit -am 'Add new feature'`
4. Push to the branch: `git push origin feature/your-feature`
5. Submit a pull request

License

This project is licensed under the ISC License - see the LICENSE file for details.

Support

For questions or issues, please open an issue on the GitHub repository or contact the development team.

Built with ❤️ using Next.js, Prisma, and Firebase

Migration Complete

✅ Migration Complete: Express → Next.js with Firebase

🎯 Summary

Your project has been successfully migrated from an Express.js backend with static HTML frontend to a modern **Next.js** application with **Firebase** as the backend.

📂 New Project Structure

```
Project 3/
├── app/                          # Next.js App Router (pages & routes)
│   ├── api/
│   │   └── orders/
│   │       ├── route.js          # POST & GET endpoints
│   │       └── [id]/route.js     # PUT endpoint for updates
│   ├── admin/
│   │   └── dashboard/page.js     # Admin order management
│   ├── rider/
│   │   └── dashboard/page.js     # Rider order pickup
│   ├── styles/
│   │   └── navbar.css            # Component-specific styles
│   ├── menu/page.js              # Menu page
│   ├── booking/page.js           # Booking reservation
│   ├── contact/page.js           # Contact form
│   ├── login/page.js             # Login page
│   ├── register/page.js          # Registration page
│   ├── layout.js                 # Root layout (global provider)
│   ├── page.js                   # Home page
│   └── globals.css               # Global styles & theme
│
├── components/
```

```
|   ├── Navbar.js                 # Navigation component
|   └── MenuCard.js               # Menu items display
|
├── lib/
|   ├── firebase.js               # Client Firebase config
|   └── firebaseAdmin.js          # Server Firebase Admin SDK
|
├── config/
|   └── serviceAccountKey.json    # Firebase credentials (secure)
|
├── .env.local                    # Environment variables
├── next.config.js                # Next.js configuration
├── jsconfig.json                 # JS path aliases
├── package.json                  # Dependencies
├── docs/
|   ├── migration/MIGRATION.md    # Migration guide
|   └── planning/TODO.md          # Implementation checklist
├── .gitignore                    # Git ignore rules
└── README.md (existing)
```

🔄 What Was Changed

| Old (Express) | New (Next.js) | Location |

|---|---|---|

| server/index.js | API Routes | app/api/orders/ |

| client/pages/*.html | React Pages | app/*/page.js |

| client/assets/js/main.js | React Components | components/ |

| config/firebaseAdmin.js | lib/firebaseAdmin.js | lib/ |

| Client JS module | lib/firebase.js | lib/ |

| Express routes | Next.js API routes | app/api/ |

✨ Key Features Added

✅ **React Components** - Reusable, interactive UI

✅ **Server-side Rendering** (optional) - Better SEO

✅ **API Routes** - Secure backend endpoints

✅ **Dark Mode Toggle** - User preference stored in localStorage

✅ **Mobile Navigation** - Responsive hamburger menu

✅ **Admin Dashboard** - View and manage orders

✅ **Rider Dashboard** - Track available orders

✅ **Form Pages** - Booking, Contact, Login, Register

✅ **Firebase Integration** - Both client and server-side

🚀 Next Steps to Complete

1. Install Dependencies

```
cd "/home/nick/Documents/web/Project 3"
npm install
```

2. Set Up Firebase

- Make sure `.env.local` has your Firebase credentials
- Ensure `config/serviceAccountKey.json` is present (⚠️ never commit!)

3. Run Development Server

```
npm run dev
```

Visit: http://localhost:3000

4. Complete These Features

- [ ] Firebase Authentication (Login/Register pages)
- [ ] Add to Cart functionality
- [ ] Form submissions to Firestore

- [ ] Real-time order updates
- [ ] Payment integration
- [ ] Deployment setup

📝 API Routes Reference

Create Order

```
POST /api/orders
Body: { userId, items, total }
Response: { orderId }
```

Get All Orders

```
GET /api/orders
Response: [{ id, userId, items, total, status, createdAt }]
```

Update Order

```
PUT /api/orders/[id]
Body: { status }
Response: { message }
```

🔒 Security Notes

⚠️ **Never commit these files:**

- `config/serviceAccountKey.json`
- `.env.local`

✅ **Already configured:**

- `.gitignore` added to prevent accidental commits
- Environment variables separated from code
- Admin SDK used only server-side

📚 Documentation Files

- **docs/migration/MIGRATION.md** - Detailed migration guide
- **docs/planning/TODO.md** - Implementation checklist
- **This file** - Quick reference

🎉 You're Ready!

The migration is complete. Your Next.js + Firebase app is ready for development. Start with the TODO list and implement the remaining features!

Need help? Check docs/migration/MIGRATION.md for detailed instructions.

Migration Guide

Project 3 - Next.js with Firebase Migration

This project has been successfully migrated from Express.js to Next.js with Firebase as the backend.

📁 Project Structure

```
app/
  ├── api/                    # API routes (migrated from Express backend)
  │   └── orders/
  │       ├── route.js        # POST /api/orders, GET /api/orders
  │       └── [id]/route.js   # PUT /api/orders/:id
  ├── admin/
  │   └── dashboard/          # Admin dashboard (manage orders)
  ├── rider/
  │   └── dashboard/          # Rider dashboard (pickup orders)
  ├── layout.js               # Root layout with Navbar
  ├── page.js                 # Home page
```

```
  ├── menu/page.js            # Menu page
  ├── booking/page.js         # Booking page
  ├── contact/page.js         # Contact page
  ├── login/page.js           # Login page
  ├── register/page.js        # Register page
  └── globals.css             # Global styles

components/
  ├── Navbar.js               # Navigation component
  └── MenuCard.js             # Menu items component

lib/
  ├── firebase.js             # Client-side Firebase config
  └── firebaseAdmin.js        # Server-side Firebase Admin SDK

config/
  └── serviceAccountKey.json  # Firebase Admin credentials (keep secure!)
```

🚀 Getting Started

Prerequisites

- Node.js 18+ installed
- Firebase project created
- `.env.local` file with Firebase credentials

Installation

1. Install dependencies:

```
npm install
```

1. Ensure `.env.local` exists with your Firebase config:

```
NEXT_PUBLIC_FIREBASE_API_KEY=your_api_key
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=your_auth_domain
NEXT_PUBLIC_FIREBASE_PROJECT_ID=your_project_id
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=your_storage_bucket
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=your_sender_id
NEXT_PUBLIC_FIREBASE_APP_ID=your_app_id
NEXT_PUBLIC_FIREBASE_MEASUREMENT_ID=your_measurement_id
FIREBASE_ADMIN_SDK_KEY=./config/serviceAccountKey.json
```

1. Ensure `config/serviceAccountKey.json` is in place (never commit this file!)

Running the Development Server

```
npm run dev
```

Visit http://localhost:3000 in your browser.

📝 What's Changed

From Express to Next.js API Routes

Old (Express):

```
app.post("/api/orders/create", async (req, res) => {
  // Handle order creation
});
```

New (Next.js):

```
// app/api/orders/route.js
export const POST = async (req) => {
  // Handle order creation
};
```

Pages Migrated

- ✅ Home page → `app/page.js`
- ✅ Menu → `app/menu/page.js`
- ✅ Booking → `app/booking/page.js`
- ✅ Contact → `app/contact/page.js`
- ✅ Login → `app/login/page.js`
- ✅ Register → `app/register/page.js`
- ✅ Admin Dashboard → `app/admin/dashboard/page.js`
- ✅ Rider Dashboard → `app/rider/dashboard/page.js`

Frontend Changes

- Static HTML pages converted to React components

- Client-side navigation using Next.js `Link` component
- Dark mode toggle using localStorage and state management
- Responsive hamburger menu

🔐 Firebase Integration

Client-side (Browser)

- Used in pages for authentication, real-time updates
- Located in: `lib/firebase.js`

Server-side (API Routes)

- Used in API routes for secure database operations
- Located in: `lib/firebaseAdmin.js`

📚 Next Steps

1. **Implement Authentication:**

- Update Login/Register pages to use Firebase Auth

- Add auth context for global state

1. **Add Cart Functionality:**

- Create a cart management system

- Update MenuCard component

1. **Complete Forms:**

- Connect Booking and Contact forms to Firebase

- Add form validation

1. **Deploy:**

bash

npm run build

```
npm start
```

📦 Build & Production

```
npm run build    # Build for production
npm start        # Start production server
npm run lint     # Run ESLint
```

🔗 Useful Links

- [Next.js Documentation](https://nextjs.org/docs)
- [Firebase Documentation](https://firebase.google.com/docs)
- [Next.js API Routes](https://nextjs.org/docs/app/building-your-application/routing/route-handlers)

Planning: Day 1 Structure

```
restaurant-system/
│
├── client/
│   ├── assets/
│   │   ├── css/
│   │   │   └── style.css
│   │   └── js/
│   │   ├── firebase.js
│   │   ├── auth.js
│   │   └── main.js
│   │
│   ├── pages/
│   │   ├── home.html
```

```
│ │ ├── menu.html
│ │ ├── booking.html
│ │ ├── contact.html
│ │ ├── login.html
│ │ ├── register.html
│
└── server/
├── index.js
└── config/
├── firebaseAdmin.js
└── serviceAccountKey.json
```

Planning: TODO List

TODO: Next.js Migration Checklist

Completed ✅

- [x] Next.js project structure setup
- [x] Package.json updated with Next.js dependencies
- [x] Environment variables configured (.env.local)
- [x] Firebase client config created (lib/firebase.js)
- [x] Firebase Admin SDK config created (lib/firebaseAdmin.js)
- [x] Layout and global styles created
- [x] Navbar component created with dark mode toggle
- [x] Home page migrated
- [x] Menu page migrated
- [x] Booking page migrated
- [x] Contact page migrated
- [x] Login page migrated

- [x] Register page migrated
- [x] Admin dashboard created
- [x] Rider dashboard created
- [x] API routes created for orders CRUD
- [x] Migration documentation created

In Progress 🔄

None

TODO - Future Implementation 📋

- [ ] Connect Login/Register to Firebase Auth
- [ ] Add Firebase Auth context for global user state
- [ ] Implement Cart functionality with localStorage
- [ ] Connect form submissions to Firestore
- [ ] Add form validation and error handling
- [ ] Payment integration (Stripe/M-Pesa)
- [ ] Real-time order updates with Firestore listeners
- [ ] Image upload functionality for menu items
- [ ] Email notifications on order status
- [ ] Deploy to Vercel/Firebase Hosting
- [ ] Set up CI/CD pipeline
- [ ] Add unit tests
- [ ] Add E2E tests
- [ ] Performance optimization
- [ ] Analytics integration

Files No Longer Needed

- [x] root-level `server.js` (reorganized under `server/index.js`)
- [x] client/pages/*.html (replaced by app/*.js pages)
- [x] client/js/*.js (reorganized under client/assets/js)
- [x] config/firebaseAdmin.js (reorganized under server/config)

Keep

- [x] config/serviceAccountKey.json (needed for server-side

Firebase access)

- [x] .env.local (for environment variables)